Name: Asher Isgitt
NetID: dal167894
Section: 501
Java Version: 25.0.2
OS: Microsoft Windows 11 Home
CPU: AMD Ryzen 7 260

AVL Tree

An AVL Tree is a binary search tree with one required invariant: for any given node, the heights of its two children must not differ by more than one. Every time a node is added to the tree, single and double rotations are executed to maintain the invariant. This keeps the tree balanced, keeping the search time at O(logn).

Insert routine: Given a key, recursively traverses down the tree starting from the root, chooses to go left or right by comparing the key to each node's key. Once it reaches the spot where the new node should be added, a new node is created. As the recursive calls return and "rise up," the balance routine is called on each node in the path going back to the root.

Search routine: Given a key, iteratively traverses down the tree starting from the root, chooses to go left or right by comparing the key to each node's key. Once a matching key is found, the function returns true. Otherwise the function returns false.

Balance routine: Given a node, checks the structure of its children to find imbalance and call single or double rotation routines if necessary. Updates the node height and returns the parent node of the newly balanced subtree once complete.

Rotate with left/right child: Given a node, rotate it with its left/right child. This involves rewiring pointers and updating heights. Once complete, the node becomes its left/right child's new right or left child. Returns the parent node of the rotated structure.

Double rotate left/right: Given a node, rotate it with left/right child. Then rotate again on the other side with the node's new child.

Splay Tree

A Splay Tree is a binary search tree with added functionality to "splay" found nodes to the root every time they are searched for. Essentially, every time a key is searched for, its node is brought all the way up the root using rotations similar to those used by an AVL tree. This ensures the tree stays relatively balanced and keeps frequently accessed nodes near the root. Generally, search operations remain around O(logn).

Insert routine: Given a key, recursively traverses down the tree starting from the root, chooses to go left or right by comparing the key to each node's key. Once it reaches the spot where the new node should be added, a new node is created.

Search routine: Given a key, iteratively traverses down the tree starting from the root, chooses to go left or right by comparing the key to each node's key. Once a matching key is found, the function splays the node to the root and returns true. Otherwise the function returns false.

Splay routine: Given a node, determines the structure of its parent and grandparent and calls zig-zig or zig-zag routines to splay the node further up. Continues this process until the node becomes the root. Returns the new root.

Rotate with left/right child: Given a node, rotate it with its left or right child. This involves rewiring pointers and updating parent pointers. Once complete, the node becomes its left or right child's new right or left child. Returns the parent node of the rotated structure.

Zig-zig left/right rotation: Used when a node, parent, and grandparent are all aligned. Given a node, rotates its parent with its grandparent, and then rotates the parent with its left/right child.

Zig-zag left/right rotation: Used when a node, parent, and grandparent are in a bent structure. Given a node, it rotates its parent with its left/right child, and then the grandparent with its right/left child.

Hash Table

A hash table ensures O(1) access time by using a hash function to translate a key value into an array index. Collisions occur when two different keys hash to the same index, and different methods can be used to resolve collisions. Chaining involves building a linked list in each index by chaining elements together when they hash to the same index. Quadratic probing involves adding a quadratically increasing value to the hashed index until an open space for the data is found. For this project, I made implementations using chaining and quadratic probing.

Division Hash function: Both implementations have the same hash function. Given a key, the hash function puts it into the following formula: **abs(key) mod tableSize**. The result is returned as the array index.

Insert routine: Given a key, first calls the ensure capacity function to make sure there is enough space to safely store the data. Then, it puts the key into the hash function and tries to add it to the resulting index. If the index is already full, it uses either chaining or quadratic probing to resolve the collision.

Search routine: Given a key, hashes the key to get the index and checks if the key is there. If not, either traverses the linked list or follows the quadratic probing path until the data is found. Returns true if found and false if not.

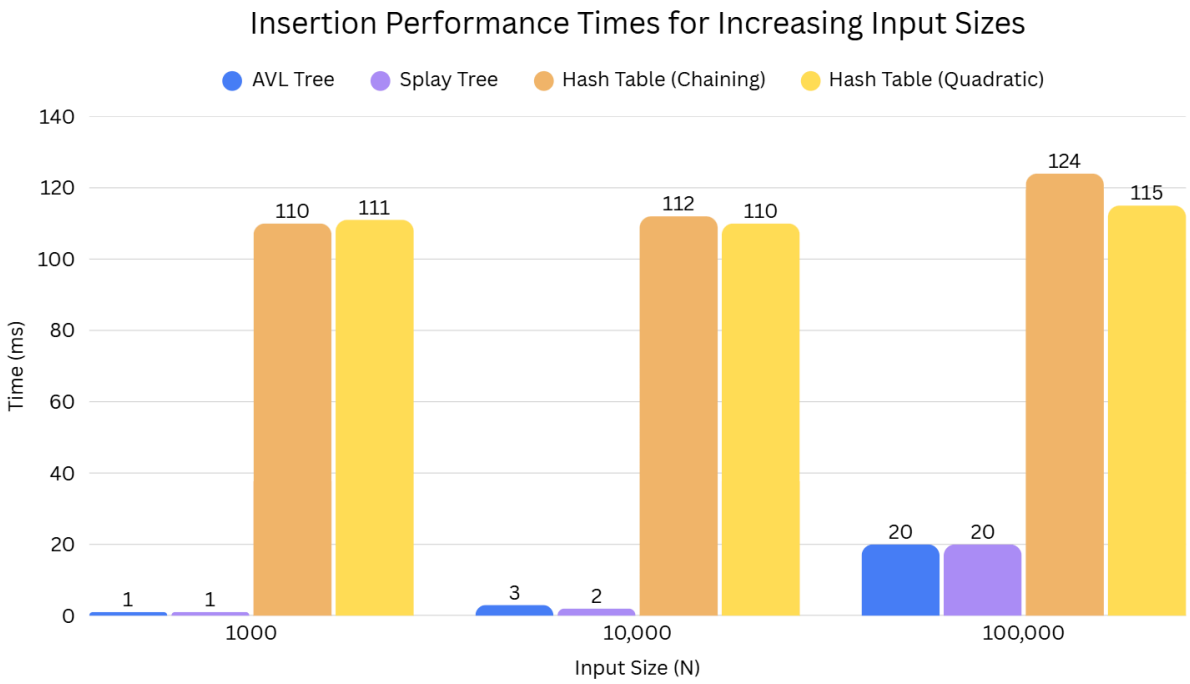
Ensure capacity routine: Once the amount of elements in the table goes above the desirable amount, this function first uses Java's Big Integer class to find the next nearest prime after doubling the array size. The array size needs to be prime to ensure a relatively even distribution

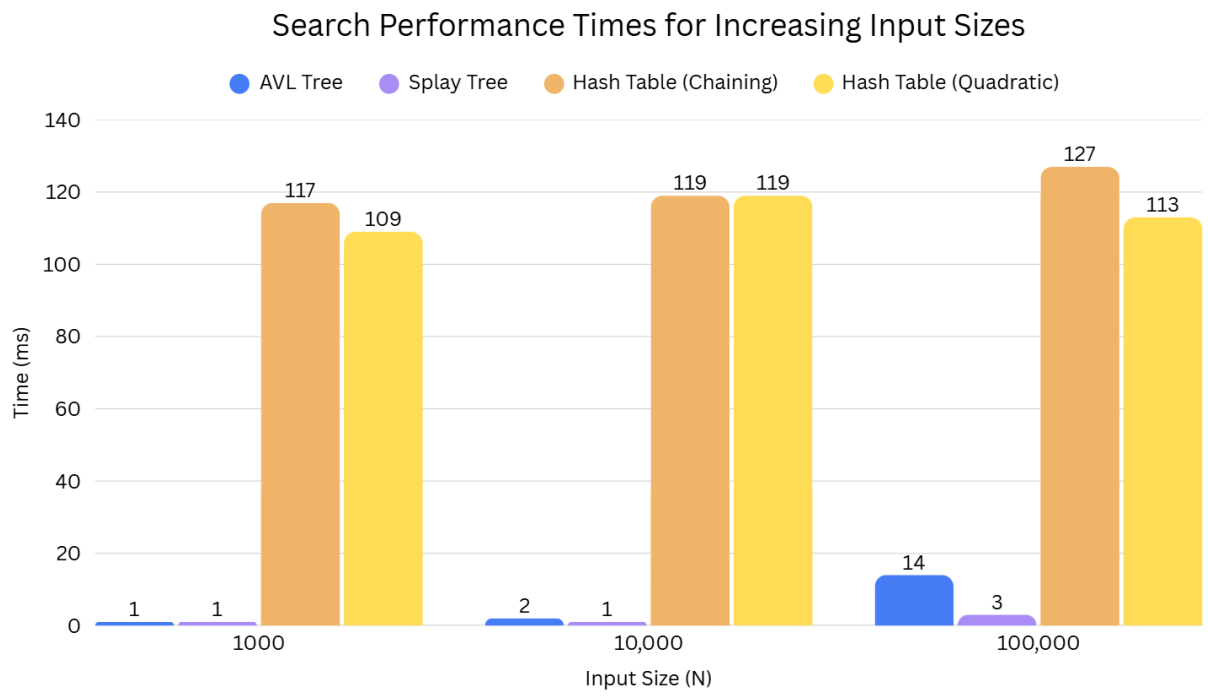
of elements with modulo division hashing. Then a new array with this prime size is created, and all elements of the old table are rehashed into the new table.

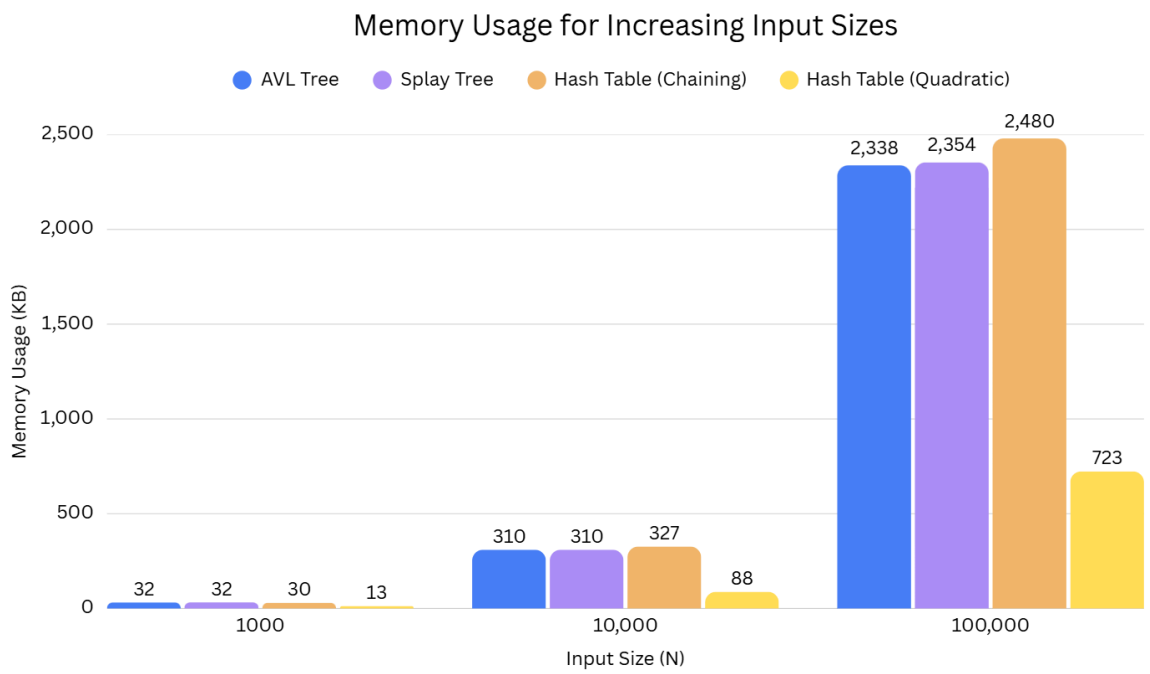
Add at index routine (chaining): In the chaining implementation, this function takes in a key and index and appends a linked list node containing the key at the end of that index's list. It checks for a duplicate and returns early without adding a new node if one exists.

Quadratic probing routine: Takes in a key and hashes it. Iteratively adds a quadratically increasing term to the base hashed index until an empty slot or duplicate is found. Returns the resulting index.

Performance Results







Discussion

The results from the testing were very interesting to me. AVL trees and Splay trees took about the exact same amounts of time for insertions at each input size, and stayed consistently close in search times until N = 100,000, where AVL searches took significantly longer. AVL and Splay Trees used almost exactly the same amounts of memory.

At first glance, the hash tables had extremely slow insert and search times which seemed strange to me. I think the slowdown may come from the slow O(N) process of resizing the table.

However it is important to notice that while at these input sizes the trees are much faster, the hash table insert and search times grew at a much slower relative rate than the tree times did. This reinforces the idea of O(1) access time, as the times are staying relatively constant even at huge input sizes. The hash table using quadratic probing turned out to be slightly faster in most cases.

In terms of memory usage, I did not include data for search routines as these are read-only and my program only output garbage memory values that were either close to 0 or negative. For insert routines, both trees and the chaining hash table used an almost identical amount of memory. This makes sense because all three had the extra memory overhead of Node objects storing pointers and data. The chaining hash table used more memory at N = 100,000, possibly due to the unused allocated array space. The quadratic probing hash table used the least amount of memory by a huge margin, because it only required an integer array with minimal overhead.

It makes sense that the Splay trees had a much faster search time than AVL trees at higher input sizes, because splaying frequently searched nodes to the root drastically reduces search time for the most commonly accessed information. The AVL tree keeps the tree balanced but does not give priority to frequently accessed nodes.

Conclusion

I suspect there may be inefficiencies in my hash table implementations due to their slow performance on display here, but I was able to show that their performance remains mostly constant at high input sizes. Overall, my performance times and memory use were extremely consistent across several trials. Splay Trees came out on top over AVL trees for insert and search times, as well as memory usage. Quadratic Probing Hash Tables used by far the least amount of memory out of all the data structures and slightly outperformed Chaining Hash Tables.